專屬 AI 財務投資系統：完整系統設計規格書 (SDS) V2.0

1. 系統架構與核心目標

本系統提供全自動化的台股投資輔助與資產管理功能。結合每日盤後自動化數據抓取、AI 雙軌標籤化（價值型/成長型），以及基於使用者偏好的「動態評分引擎」，打造專屬理財儀表板。

- 後端與前端：Antigravity (Python 3.10+)。
- 資料庫：PostgreSQL (負責高階運算與事務處理)。
- 資料來源：台灣證交所 (TWSE) OpenAPI。
- 關鍵機制：ETL 自動排程、Header/Detail 投資組合雙表同步、Admin 身分模擬與稽核。

---

2. 資料庫建置腳本 (PostgreSQL Schema)

系統採用關聯式設計，並運用 JSONB 提升個人化設定的擴充性。

SQL

```sql
-- 啟用 UUID 生成擴充功能
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. 使用者基本資料表
CREATE TABLE Users (
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) DEFAULT 'user' CHECK (role IN ('user', 'admin')),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. 使用者 AI 評分權重偏好表 (JSONB 格式)
CREATE TABLE User_Preferences (
  user_id UUID PRIMARY KEY REFERENCES Users(user_id) ON DELETE CASCADE,
  weight_map JSONB NOT NULL,
```

```sql
  -- 範例格式: {"value": {"yield_w": 0.6, "pe_w": 0.4}, "growth": {"yoy_w": 0.7, "roe_w": 0.3}}
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. 股票靜態基本資料表
CREATE TABLE Stocks (
  ticker VARCHAR(20) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  industry VARCHAR(100)
);

-- 4. 每日市場數據與財務特徵表 (ETL 寫入目標)
CREATE TABLE Market_Data (
  ticker VARCHAR(20) REFERENCES Stocks(ticker) ON DELETE CASCADE,
  date DATE NOT NULL,
  pe_ratio NUMERIC(10, 4),
  yield NUMERIC(10, 4),
  yoy_growth NUMERIC(10, 4),
  roe NUMERIC(10, 4),
  ai_tag VARCHAR(50), -- 'Value' 或 'Growth'
  PRIMARY KEY (ticker, date)
);

-- 5. 投資組合：交易明細表 (Detail)
CREATE TABLE Portfolio_Transactions (
  tx_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES Users(user_id) ON DELETE CASCADE,
  ticker VARCHAR(20) REFERENCES Stocks(ticker),
  trade_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  action VARCHAR(10) NOT NULL CHECK (action IN ('BUY', 'SELL')),
  price NUMERIC(15, 4) NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0)
);

-- 6. 投資組合：持股總覽表 (Header)
CREATE TABLE Portfolio_Summary (
  summary_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES Users(user_id) ON DELETE CASCADE,
  ticker VARCHAR(20) REFERENCES Stocks(ticker),
  total_shares INTEGER NOT NULL CHECK (total_shares >= 0),
  average_cost NUMERIC(15, 4) NOT NULL,
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, ticker)
```

```
);

-- 7. 稽核日誌表 (Admin 身分模擬防護)
CREATE TABLE Audit_Logs (
  log_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_user_id UUID REFERENCES Users(user_id),
  target_user_id UUID REFERENCES Users(user_id),
  action_detail TEXT NOT NULL,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

---

3. 每日排程 ETL 與 AI 標籤化 (Python 腳本)

此腳本負責每日 16:30 抓取資料、整併特徵、過濾標籤，並透過 UPSERT 寫入資料庫確保冪等性。

Python

```python
import requests
import pandas as pd
import psycopg2
import time
from datetime import datetime
import sys

DB_CONFIG = {"dbname": "db", "user": "usr", "password": "pwd", "host": "localhost", "port": "5432"}

def fetch_data_with_retry(url, max_retries=2, wait_time=600):
  """具備 10 分鐘重試機制的 API 請求"""
  for attempt in range(max_retries):
    try:
      response = requests.get(url, timeout=30)
      response.raise_for_status()
      return response.json()
    except requests.exceptions.RequestException as e:
      if attempt < max_retries - 1:
        time.sleep(wait_time)
      else:
        sys.exit(f"嚴重錯誤，通知 Admin: {e}")
```

```
def transform_and_tag(quotes, pe_data, financials):
    """Pandas 資料整併與 AI 絕對門檻標籤化"""
    df_q = pd.DataFrame(quotes)[['Code', 'Name', 'ClosingPrice']].rename(columns={'Code': 'ticker',
'ClosingPrice': 'close_price'})
    df_pe = pd.DataFrame(pe_data)[['Code', 'PEratio', 'DividendYield']].rename(columns={'Code':
'ticker', 'PEratio': 'pe_ratio', 'DividendYield': 'yield'})
    df_f = pd.DataFrame(financials)[['Code', 'YoY_Growth', 'ROE']].rename(columns={'Code': 'ticker',
'YoY_Growth': 'yoy_growth', 'ROE': 'roe'})

    df = pd.merge(df_q, df_pe, on='ticker', how='inner')
    df = pd.merge(df, df_f, on='ticker', how='left')
    df['date'] = datetime.today().date()

    for col in ['pe_ratio', 'yield', 'yoy_growth', 'roe']:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    df['ai_tag'] = None
    df.loc[(df['pe_ratio'] < 15) & (df['yield'] > 5.0), 'ai_tag'] = 'Value'
    df.loc[(df['yoy_growth'] > 20.0) & (df['roe'] > 20.0), 'ai_tag'] = 'Growth'

    return df.where(pd.notnull(df), None)

def upsert_to_db(df):
    """使用 Raw SQL 進行冪等性寫入"""
    sql = """
        INSERT INTO Market_Data (ticker, date, pe_ratio, yield, yoy_growth, roe, ai_tag)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
        ON CONFLICT (ticker, date) DO UPDATE SET
            pe_ratio = EXCLUDED.pe_ratio, yield = EXCLUDED.yield,
            yoy_growth = EXCLUDED.yoy_growth, roe = EXCLUDED.roe, ai_tag = EXCLUDED.ai_tag;
    """
    with psycopg2.connect(**DB_CONFIG) as conn:
        with conn.cursor() as cur:
            for row in df.itertuples(index=False):
                cur.execute(sql, (row.ticker, row.date, row.pe_ratio, row.yield, row.yoy_growth, row.roe,
row.ai_tag))
        conn.commit()

if __name__ == "__main__":
    q_data =
fetch_data_with_retry("https://openapi.twse.com.tw/v1/exchangeReport/STOCK_DAY_ALL")
    pe_data = fetch_data_with_retry("https://openapi.twse.com.tw/v1/exchangeReport/BWIBBU_ALL")
    f_data = [{"Code": "2330", "YoY_Growth": "25.5", "ROE": "28.0"}] # Mock data
```

```
    final_df = transform_and_tag(q_data, pe_data, f_data)
    upsert_to_db(final_df)
```

---

4. 個人化動態評分 (高階 PostgreSQL 語法)

當使用者載入首頁時，後端不再拉取大批資料進 Python 運算，而是直接發送以下 SQL 讓資料庫透過 ->> 運算子萃取 JSONB 權重並計算總分。

- 數學公式邏輯：$TotalScore = (Yield \times W_1) + ((100 - PE) \times W_2)$（假設 PE 越低分數越高）

SQL

```sql
-- 價值型策略動態計分與排序範例
SELECT
    m.ticker,
    s.name,
    m.pe_ratio,
    m.yield,
    -- 核心：從 JSONB 取出權重字串，轉型為 NUMERIC 後與行情數據進行運算
    ROUND(
        (m.yield * CAST(up.weight_map->'value'->>'yield_w' AS NUMERIC)) +
        ((100 - m.pe_ratio) * CAST(up.weight_map->'value'->>'pe_w' AS NUMERIC)),
    2) AS personalized_score
FROM Market_Data m
JOIN Stocks s ON m.ticker = s.ticker
JOIN User_Preferences up ON up.user_id = %s -- 由 Python 後端傳入當前(或被模擬)的 user_id
WHERE m.date = CURRENT_DATE
  AND m.ai_tag = 'Value'
ORDER BY personalized_score DESC
LIMIT 5;
```

---

5. 應用程式業務邏輯規範

A. 投資組合記帳同步 (Transaction Logic)

寫入交易時，後端必須嚴格執行資料庫事務 (ACID)：

1. BEGIN
2. INSERT 至 Portfolio_Transactions。
3. UPSERT 至 Portfolio_Summary (計算新均價：總成本 / 總股數)。
4. 若任一步驟出錯（如股數扣除後小於 0），觸發 ROLLBACK；否則 COMMIT。

B. 管理員身分模擬 (Impersonation Flow)

1. 驗證與啟動：確認發起者 role == 'admin'。在 Session 中設置 is_impersonating = True 與 target_user_id。
2. 資料攔截：後端所有對 Portfolio 或 Preferences 的資料庫請求，自動將查詢條件中的 user_id 替換為 target_user_id。
3. 稽核強制寫入：若 Admin 點擊「儲存權重」或「新增交易」，後端必須在同一個 Transaction 中寫入一筆資料至 Audit_Logs。

5. 前端 UI 組件與 API 路由對照表

這份對照表採用了標準的 RESTful API 設計風格，並融入了我們專屬的 Admin 身分模擬與動態 AI 計分邏輯。

A.系統認證與 Admin 管理模組

UI 組件名稱	對應 API 路由	方法	功能描述與資料流
LoginForm (登入表單)	/api/auth/login	POST	驗證帳密。成功後建立 Session 並回傳使用者基本資料與 Role (一般使用者或 Admin)。

AdminSwitcher (頂部身分切換下拉選單)	/api/admin/impersonate	POST	僅 Admin 可見。送出目標 target_user_id，後端將 Session 標記為模擬狀態。成功後前端重新載入整個 Dashboard。
AdminSwitcher (結束模擬按鈕)	/api/admin/impersonate	DELETE	清除 Session 中的模擬標記，將視角與權限切換回 Admin 本人。

B.AI 推薦與首頁儀表板

UI 組件名稱	對應 API 路由	方法	功能描述與資料流

Dashboard (首頁排行榜視圖)	/api/recommendations	GET	透過 Query String (如 ?strategy=value) 請求推薦名單。後端會讀取當前 Session 的使用者權重，透過高階 SQL 即時計算總分並回傳前 5 名。
StockCard (單檔股票資訊卡)	/api/stocks/{ticker}	GET	點擊卡片時，載入該股票的歷史價格趨勢與詳細財報數據 (供圖表渲染使用)。
PreferenceEditor (自訂權重滑桿介面)	/api/preferences	GET	取得使用者目前設定的 AI 權重比例，用於初始化滑桿數值。

PreferenceEditor (儲存權重按鈕)	/api/preferences	PUT	提交新的權重 JSONB。若處於 Admin 模擬狀態，後端需同步觸發寫入 Audit_Logs。

C.個人投資組合管理 (Header / Detail)

UI 組件名稱	對應 API 路由	方法	功能描述與資料流
PortfolioOverview (資產總覽表格)	/api/portfolio/summary	GET	直接撈取 Portfolio_Summary (Header 表)，快速渲染使用者目前持有的各檔股票總股數與平均成本。
TransactionHistory	/api/portfolio/transactions	GET	撈取 Portfolio_Transactions (Detail 表)，提供帶有分頁功能的歷史買

(交易明細列表)			賣流水帳。
TradeModal (新增買賣紀錄彈出視窗)	/api/portfolio/transactions	POST	送出買/賣、單價與股數。**後端啟動 Transaction，寫入 Detail 並 UPSERT Header 表。**若成功，前端觸發更新 PortfolioOverview 的資料。
TradeModal (刪除錯誤紀錄按鈕)	/api/portfolio/transactions/{tx_id}	DELETE	刪除單筆明細。後端同樣需啟動 Transaction，重新加總剩餘明細並更新 Header 表的總股數與均價。

開發實作提醒 (給 Antigravity 的提示)

1. Session 攔截器 (Middleware)：在後端實作時，針對 `/api/recommendations` 與 `/api/portfolio/*` 等路由，必須加上一個 Middleware。這個 Middleware 會檢查目前 Session 是否有 `is_impersonating = True`，如果有，就強制把即將進入 SQL 查詢

的 `user_id` 替換成被模擬的目標 ID。

2. 前端狀態管理 (State Management)：前端建議使用全域狀態 (如 React Context 或 Redux) 來記錄「目前正在模擬誰」。這樣當 Admin 切換身分時，前端 UI 就能顯示明顯的提示（例如：畫面頂部出現一條紅底白字的警告列：「您目前正在以 User_A 的身分操作系統」），避免 Admin 改錯資料。